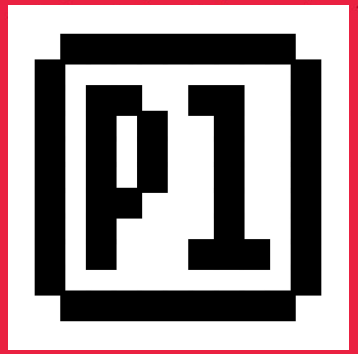
Memoria



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de **Río Negro**

13 / 2026

r33


ドドドド

Segmentos

UNRN Universidad Nacional de Río Negro

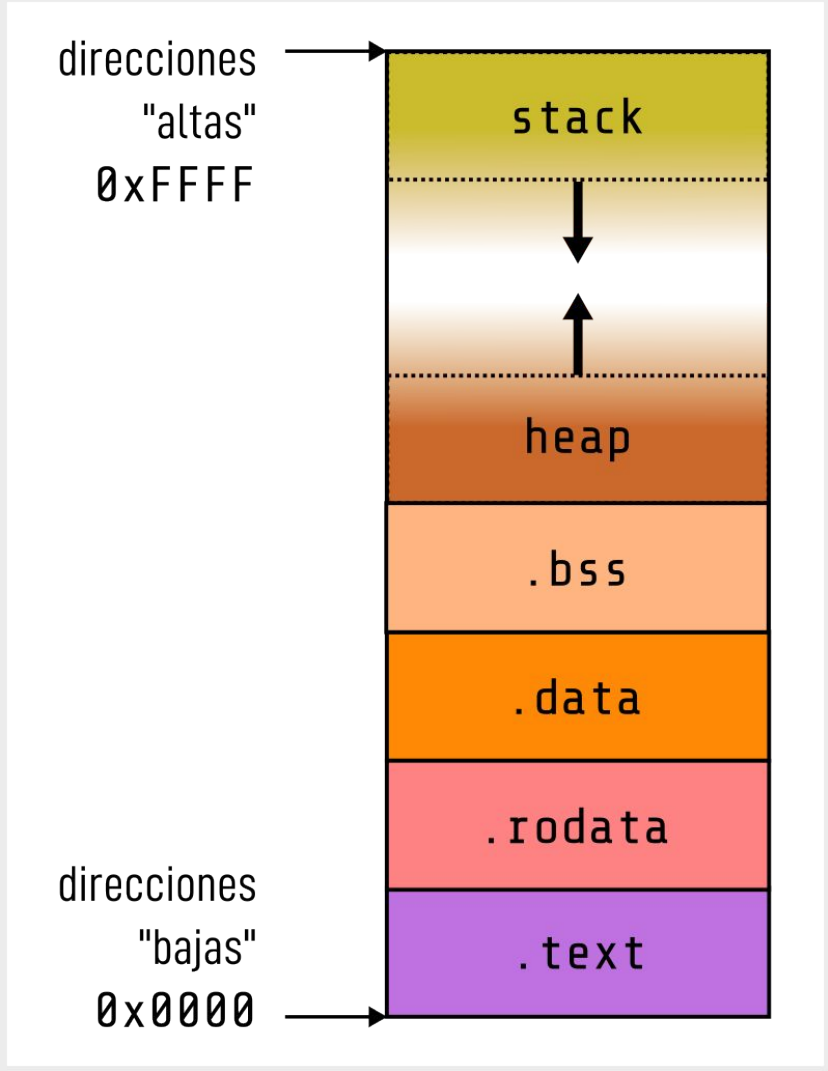
Todas las variables tienen un lugar en la memoria

Cada uno tiene características *específicas*

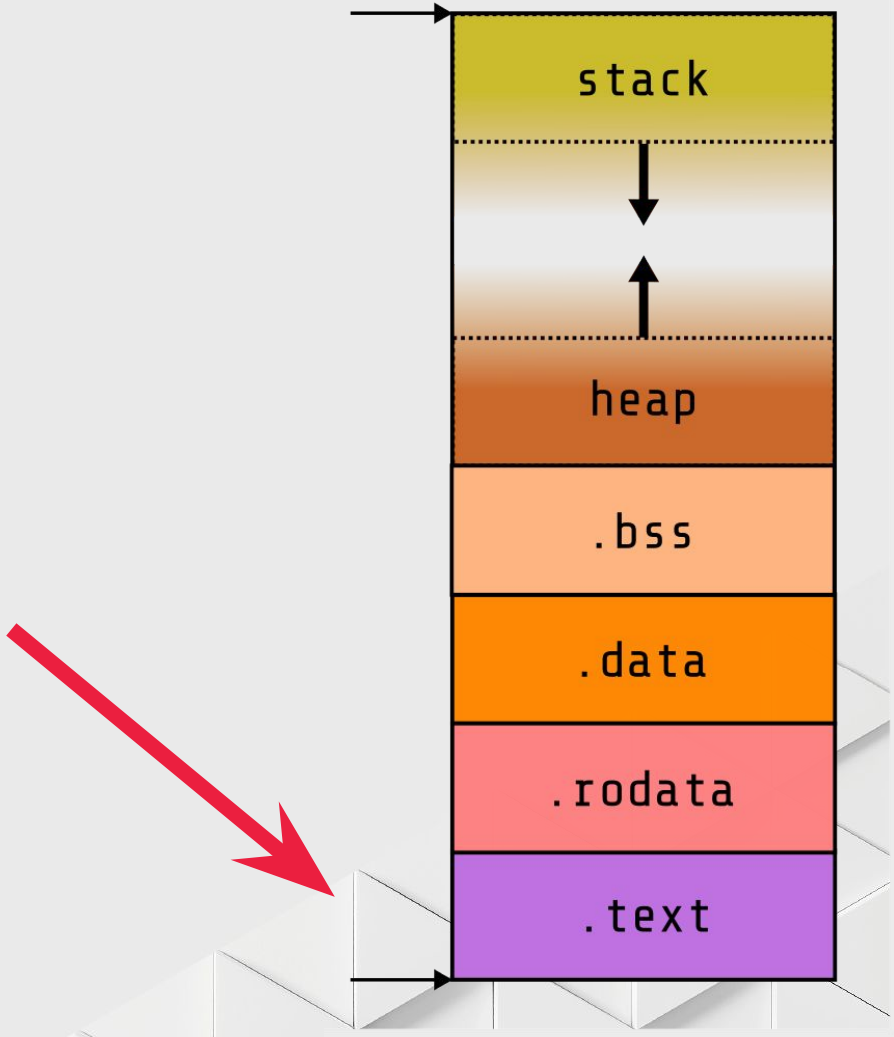
En donde se alojan las variables

El mapa de la memoria

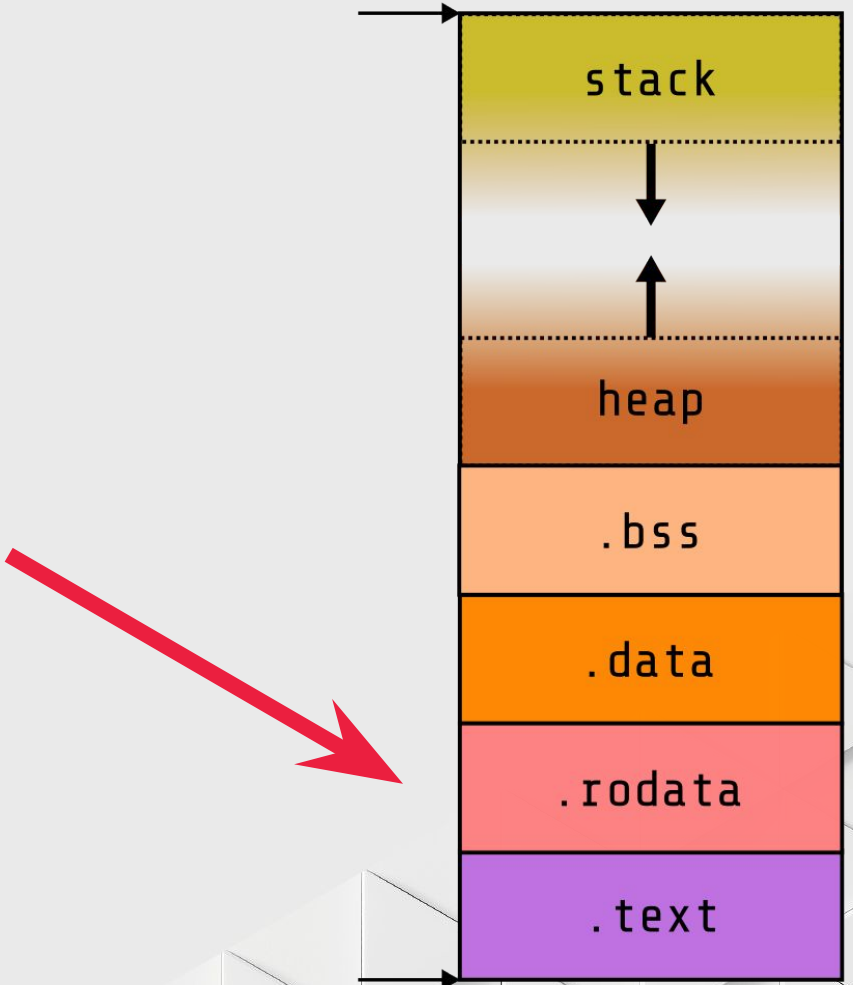
de un programa



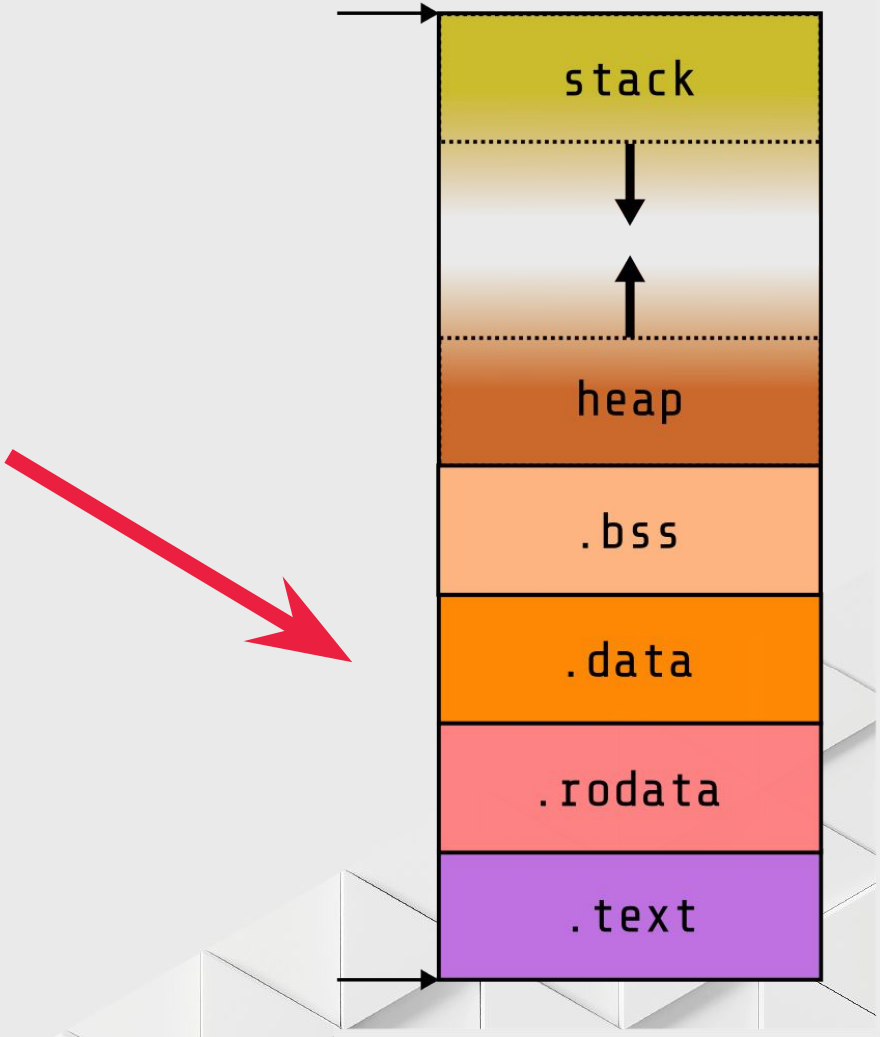
El código del programa



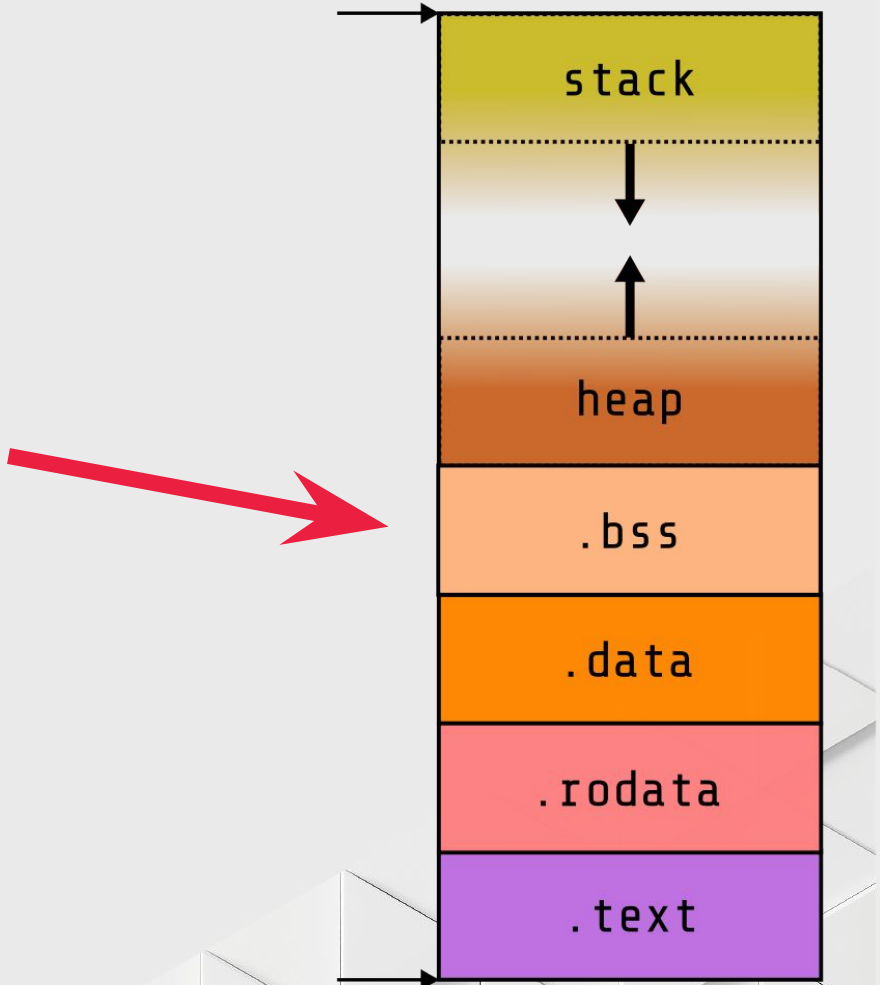
Constantes y literales de cadenas



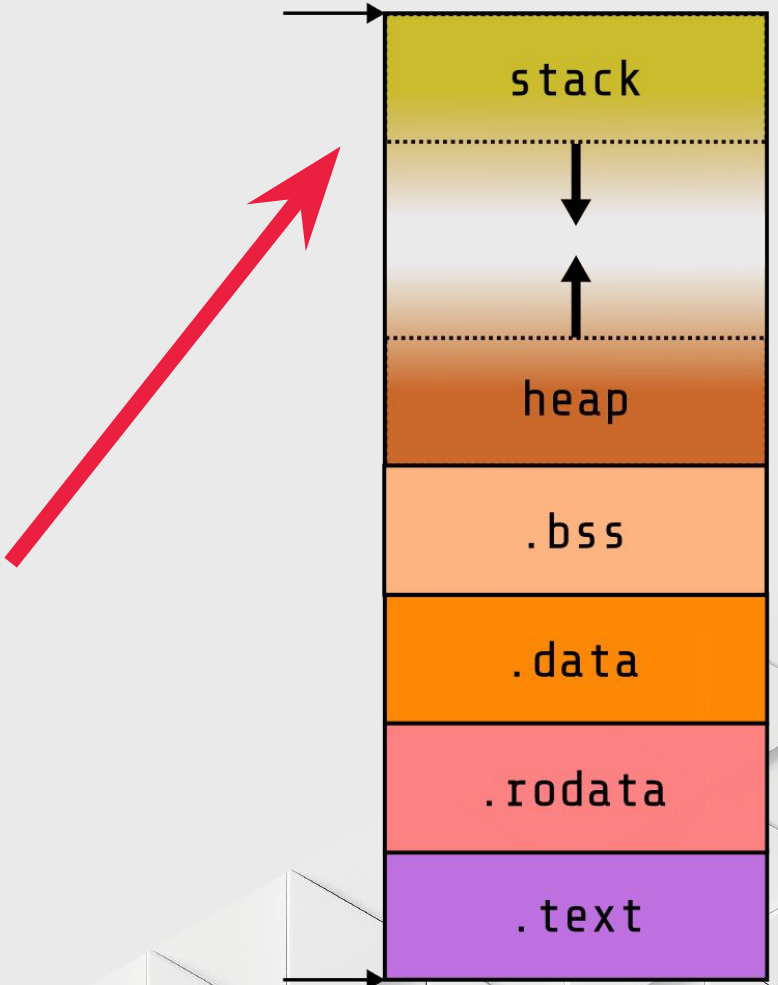
variables globales



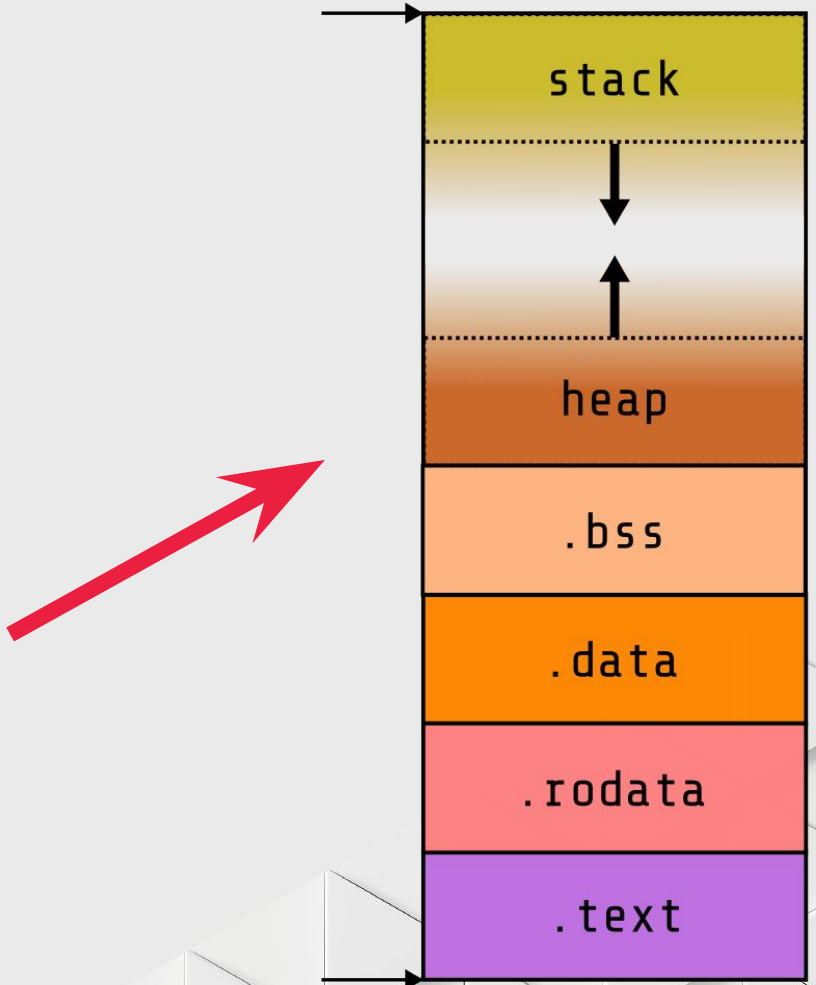
variables estáticas



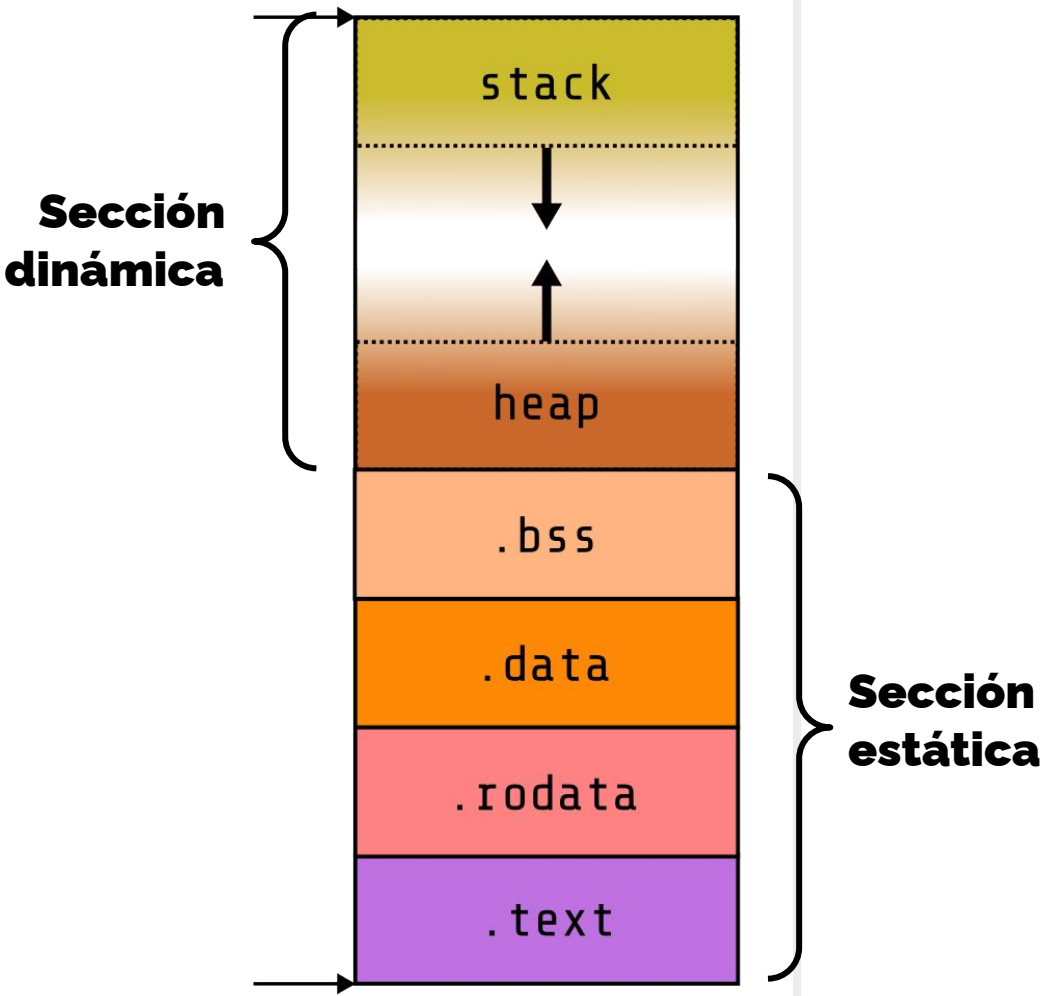
memoria automática

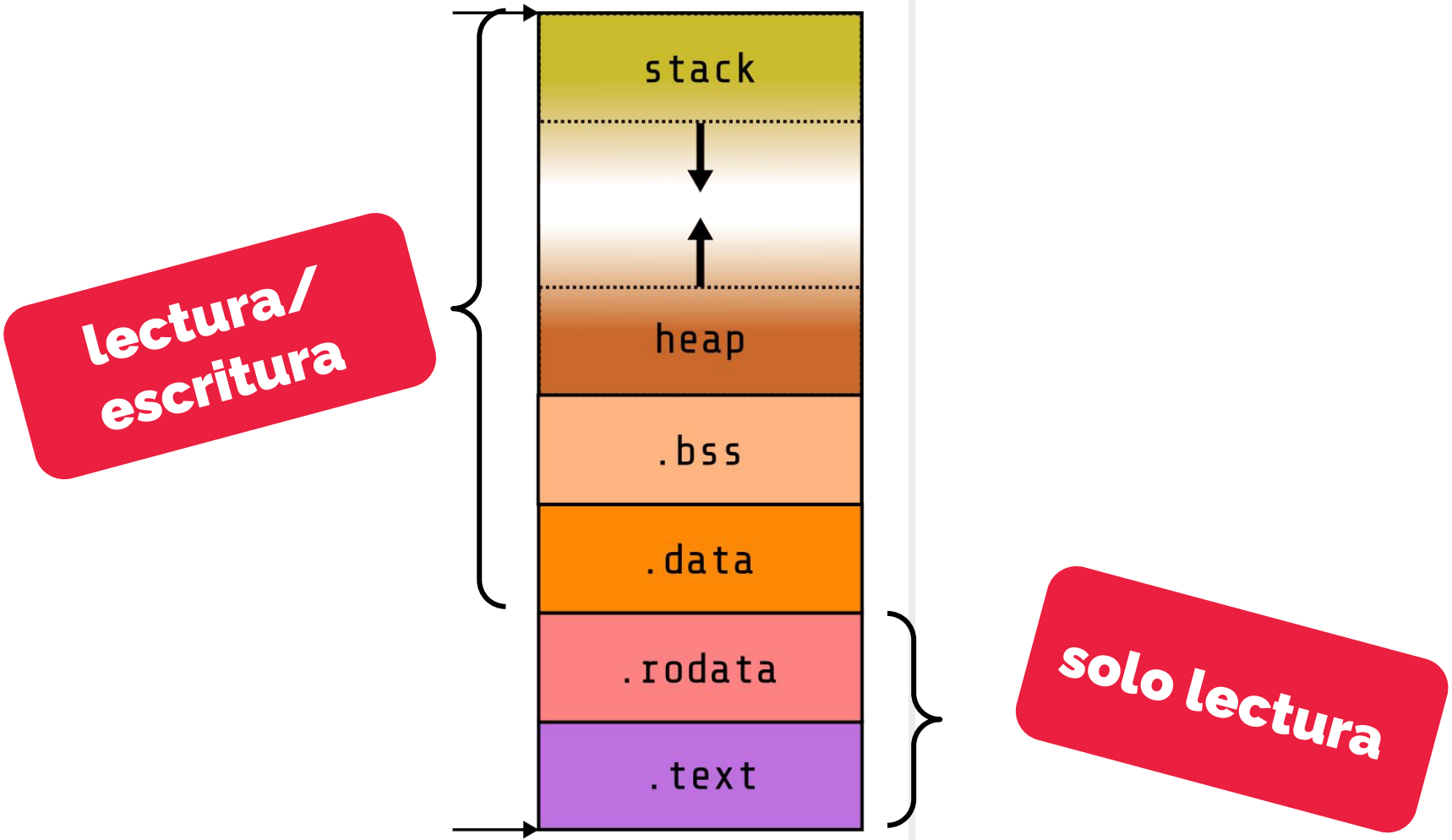


memoria dinámica



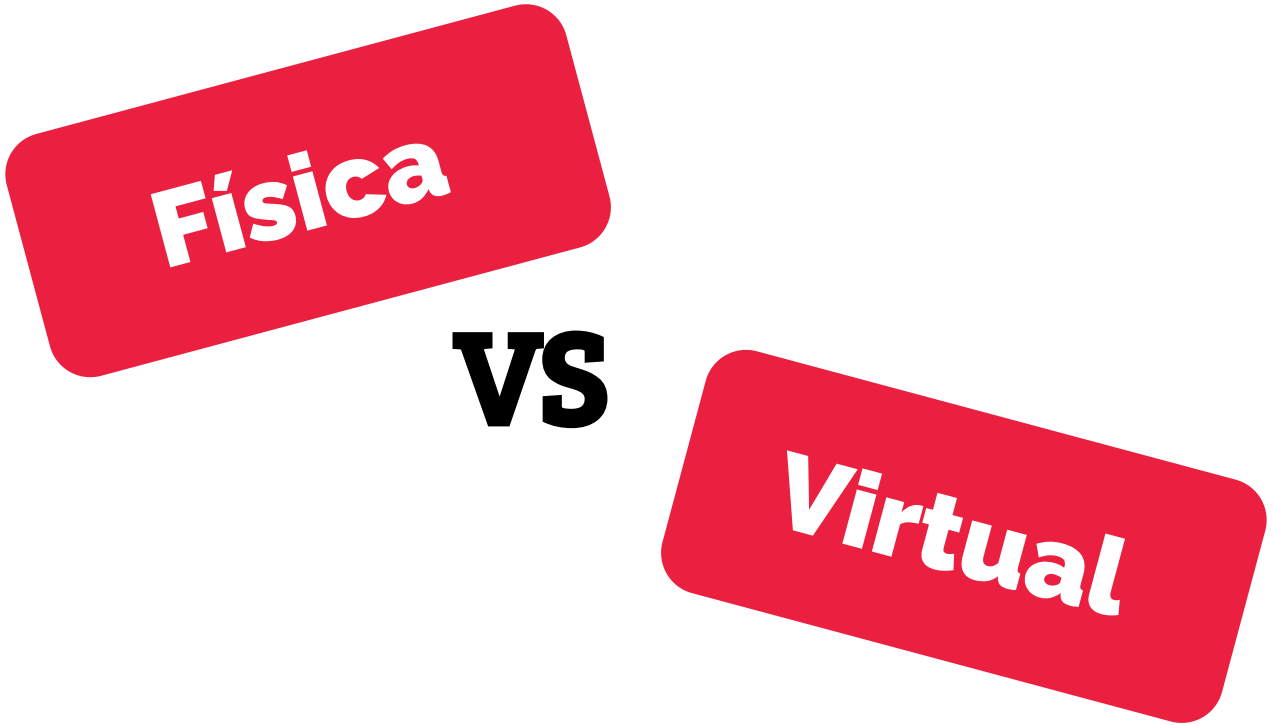

¿Preguntas?


Sección dinámica
stack
heap
.bss
.data
.rodata
.text
Sección estática


lectura/
escritura
stack
heap
.bss
.data
.rodata
.text
solo lectura

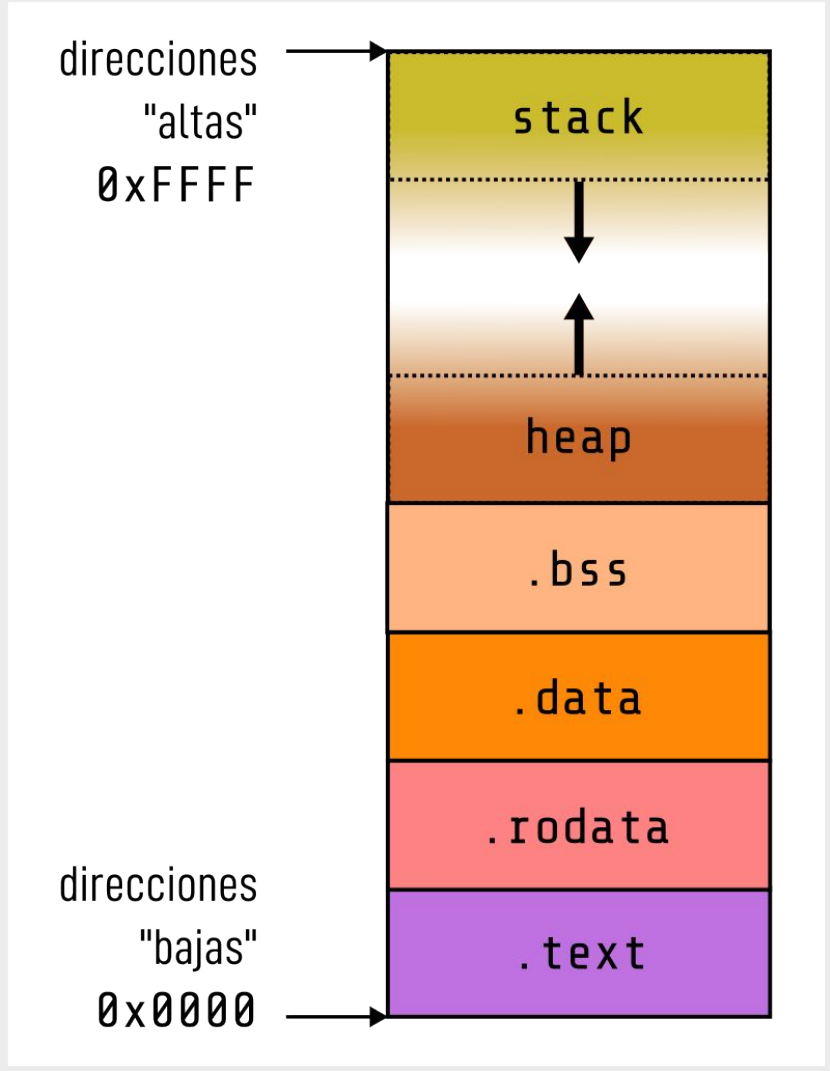

¿Preguntas?

Memoria



Pero...

¿Es realmente cero?



Física

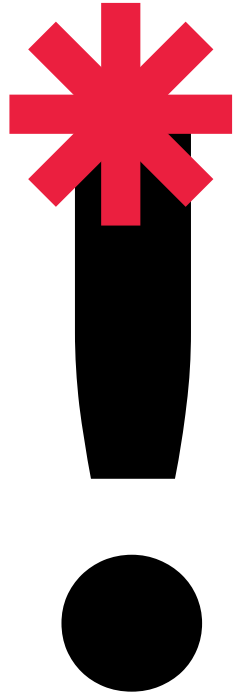
El hardware de RAM en la computadora, único, compartido y limitado

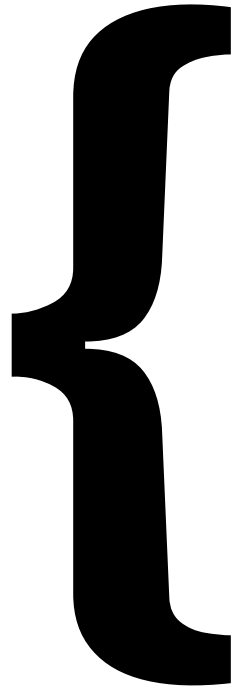
Virtual

Un mapa privado y aislado para cada programa.
El `0` de un programa, es *cualquier* dirección física (ASLR + páginas del SO).

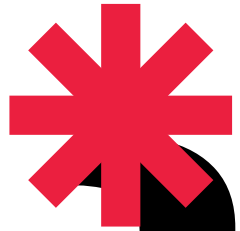


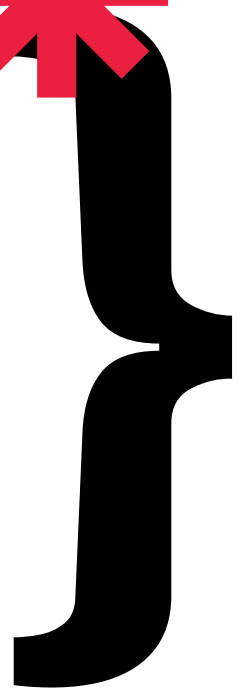
No siempre es igual





Van a ver este tema en Sistemas Operativos






¿Preguntas?

Sección dinámica

UNRN Universidad Nacional de Río Negro

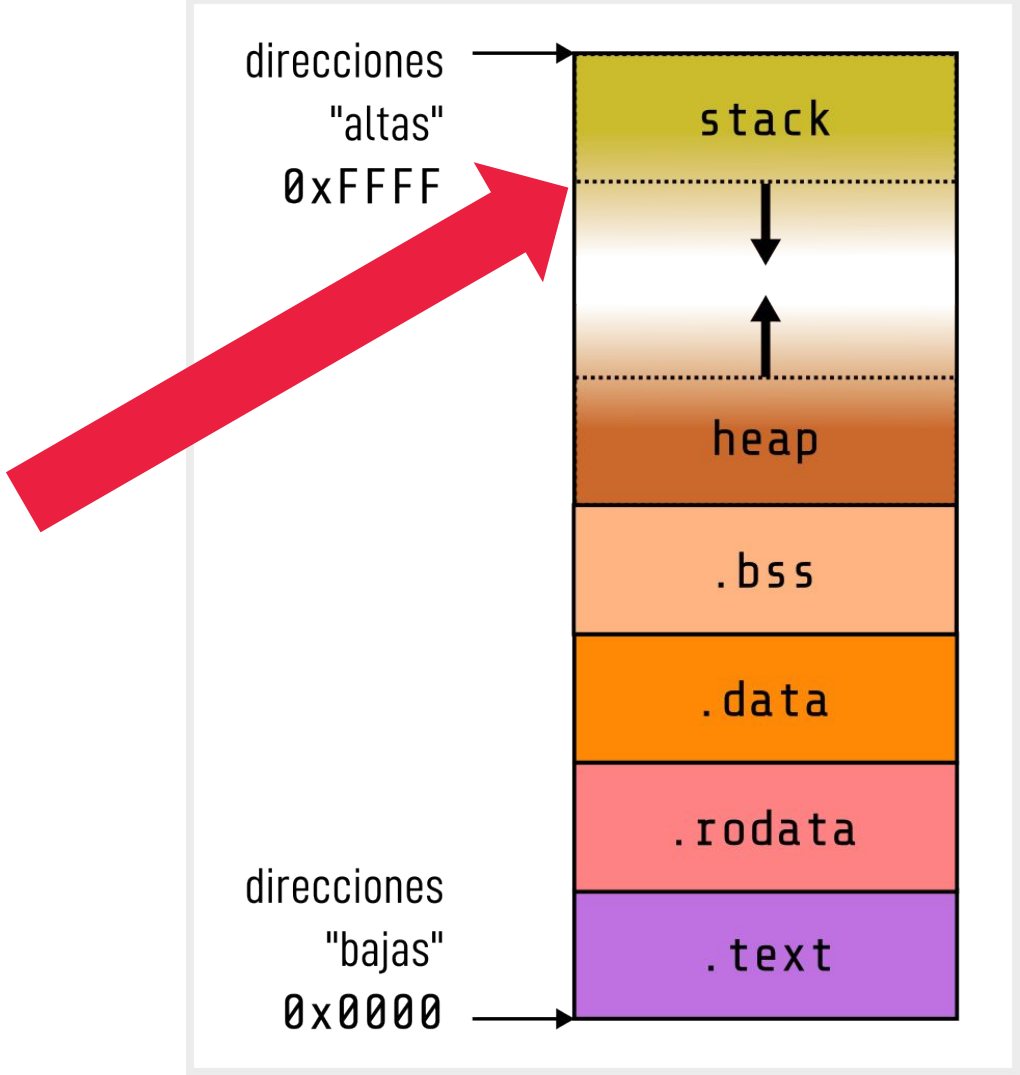
de Stack

o automática



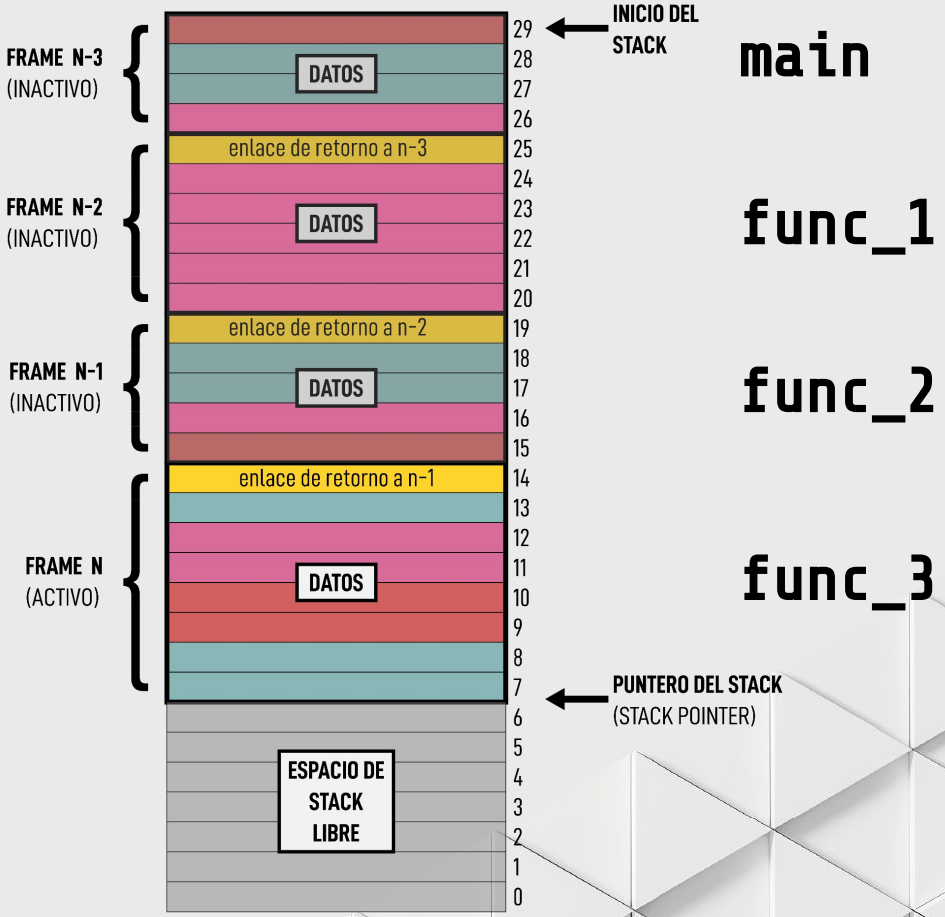
UNRN Universidad Nacional de Río Negro

UNRN Universidad Nacional de Río Negro


direcciones
"altas"
0xFFFF
stack
heap
.bss
.data
.rodata
.text
direcciones
"bajas"
0x0000

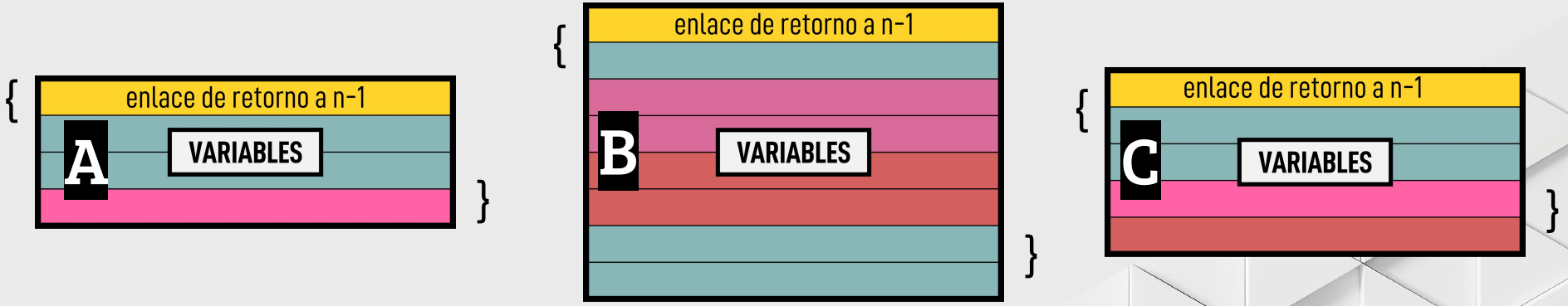
1 alta eficiencia

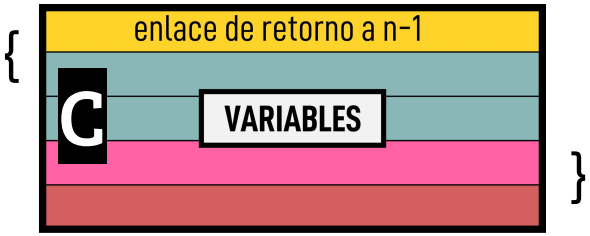
No hay 'huecos'



2 de *reutilización* continua

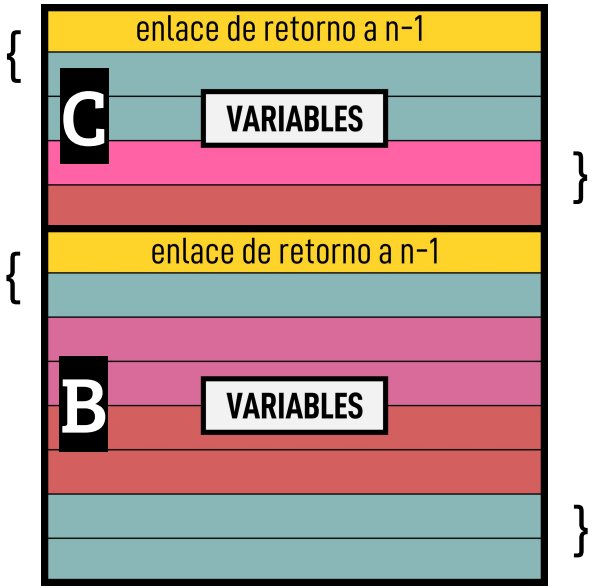
Sean 3 funciones





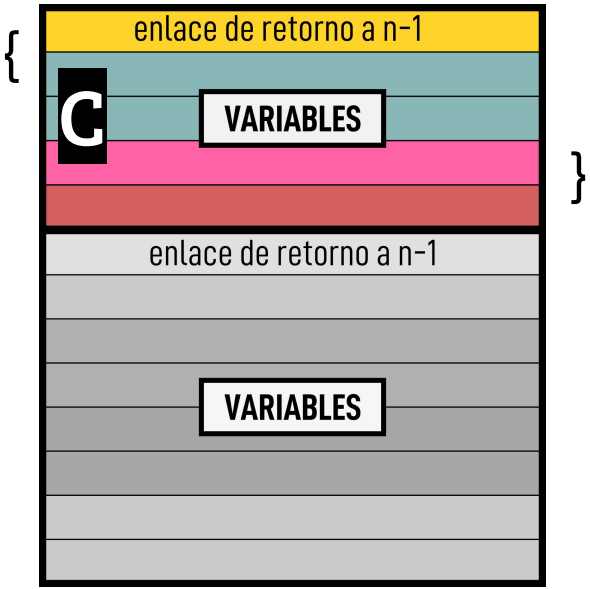
1

llamamos a C



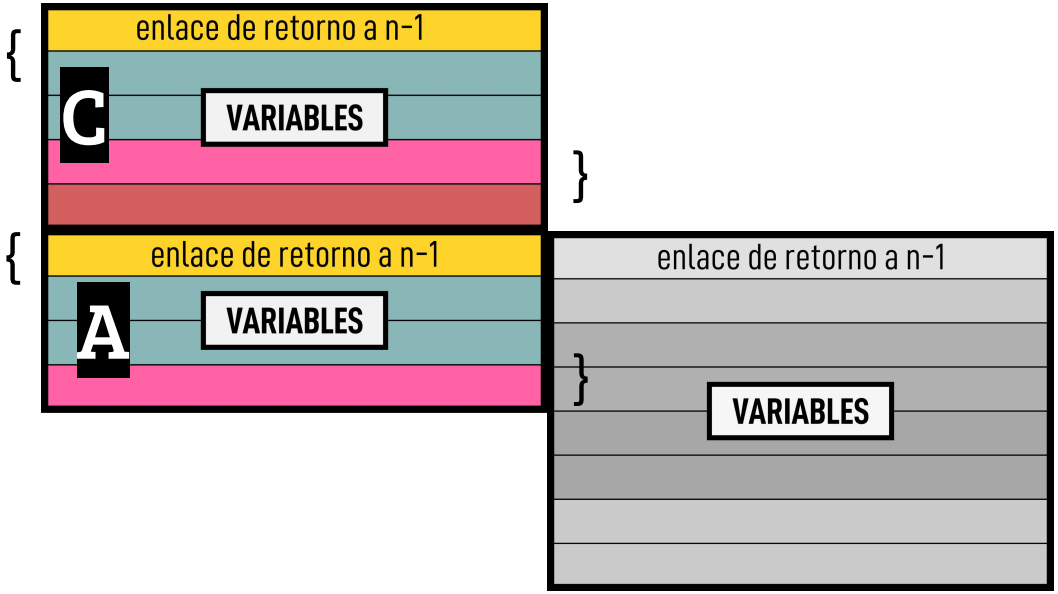
2

llamamos a B


{
enlace de retorno a n-1
C
VARIABLES
}
enlace de retorno a n-1
VARIABLES

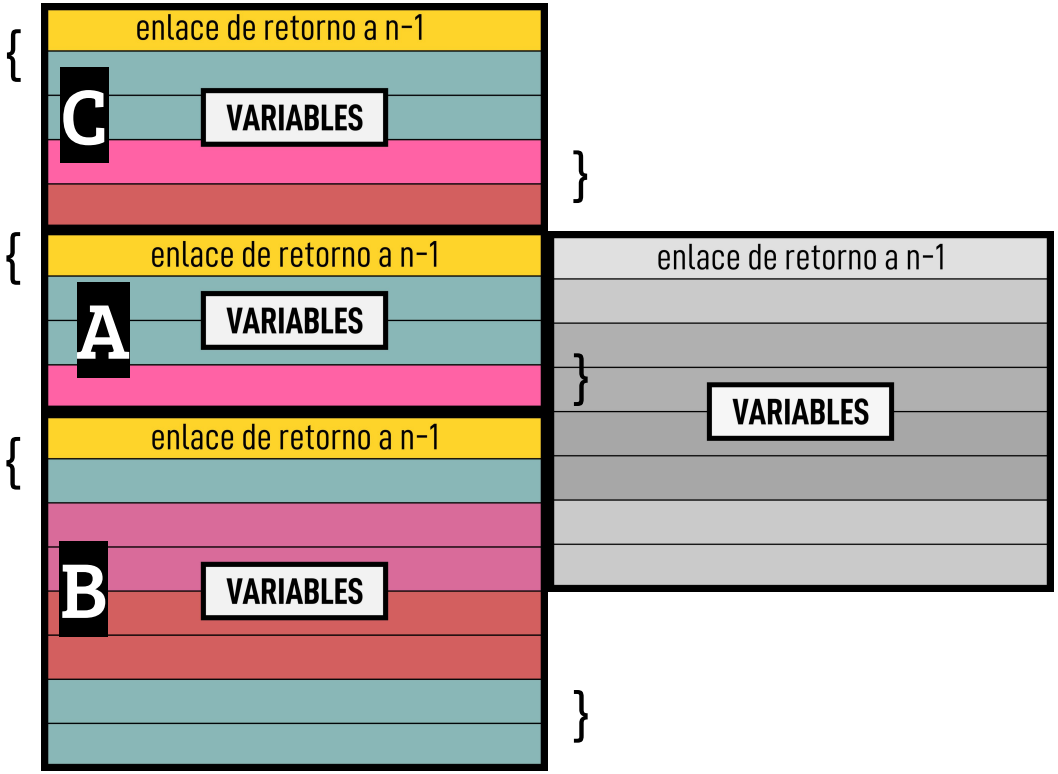
3

B finaliza



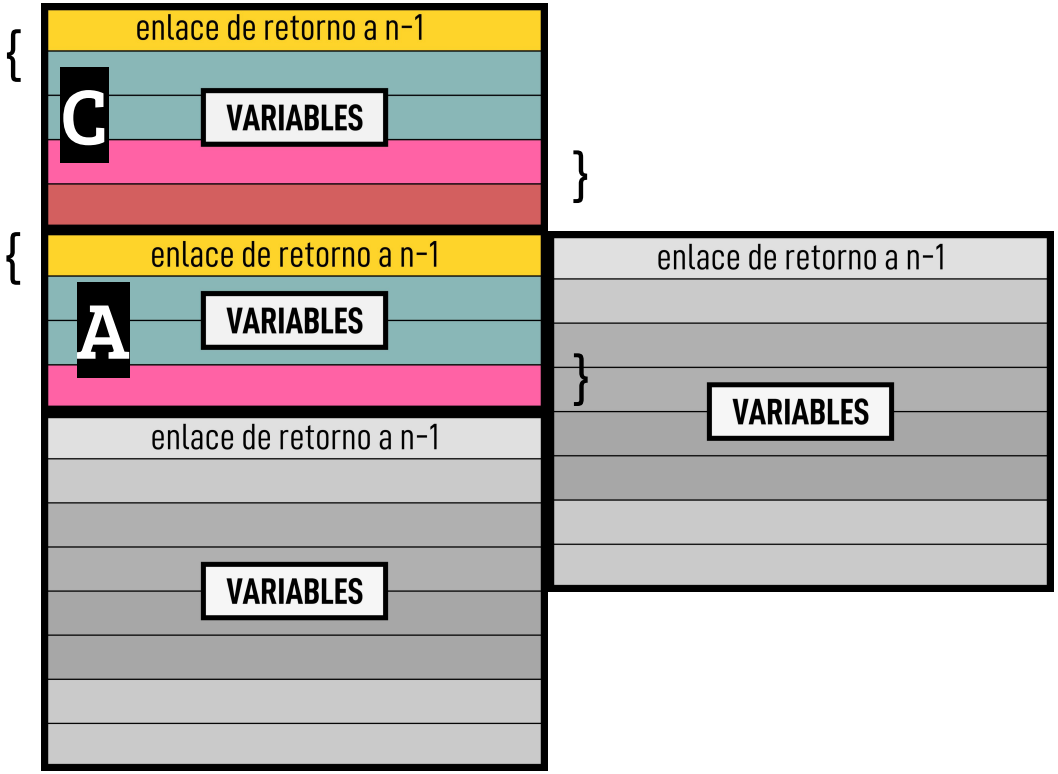
4

llamamos a A



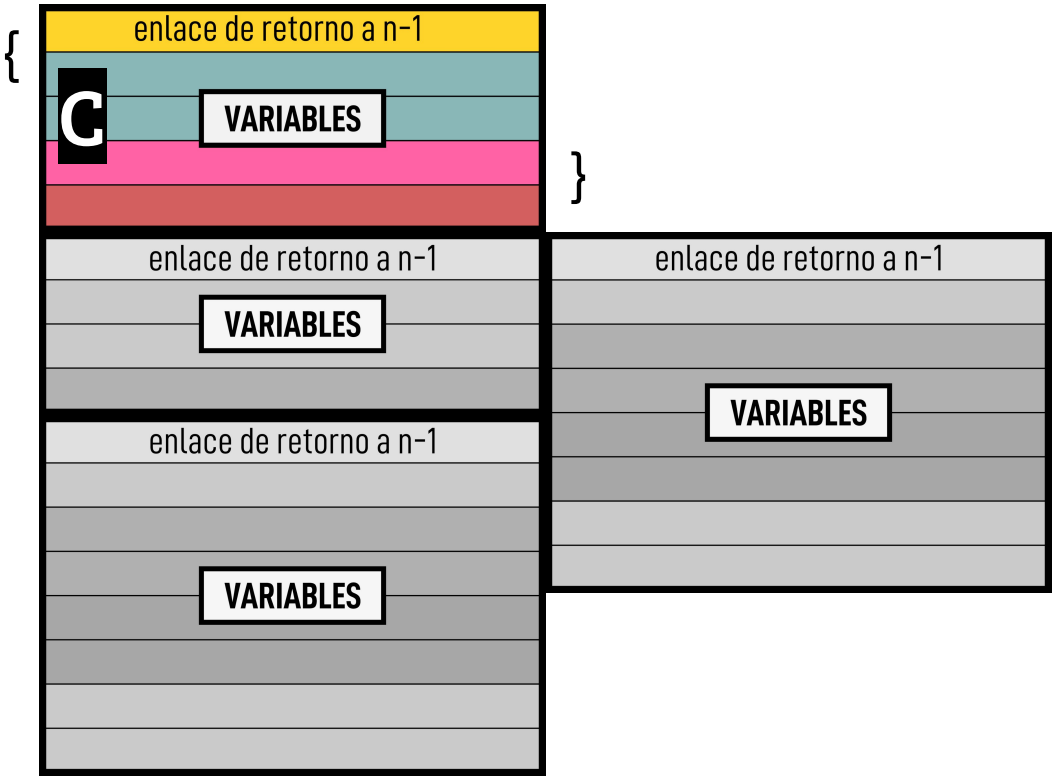
5

llamamos a B


enlace de retorno a n-1
C
VARIABLES
enlace de retorno a n-1
A
VARIABLES
enlace de retorno a n-1
VARIABLES
enlace de retorno a n-1
VARIABLES

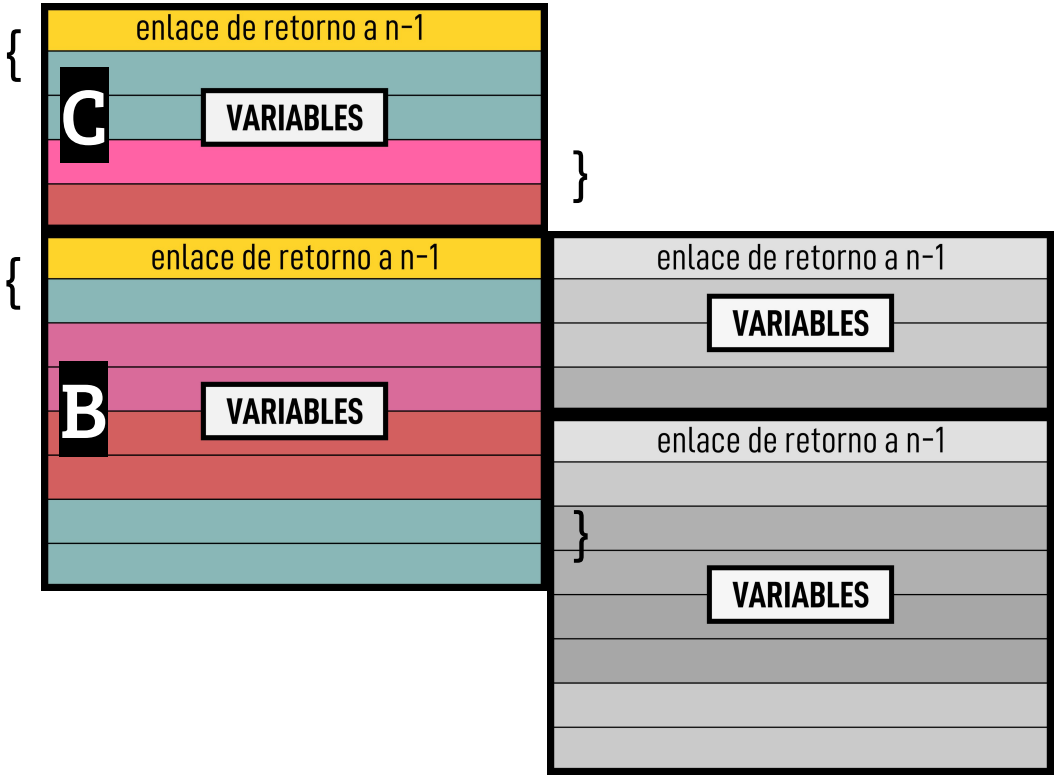
6

B finaliza


{
enlace de retorno a n-1
C
VARIABLES
}
enlace de retorno a n-1
VARIABLES
enlace de retorno a n-1
VARIABLES
enlace de retorno a n-1
VARIABLES

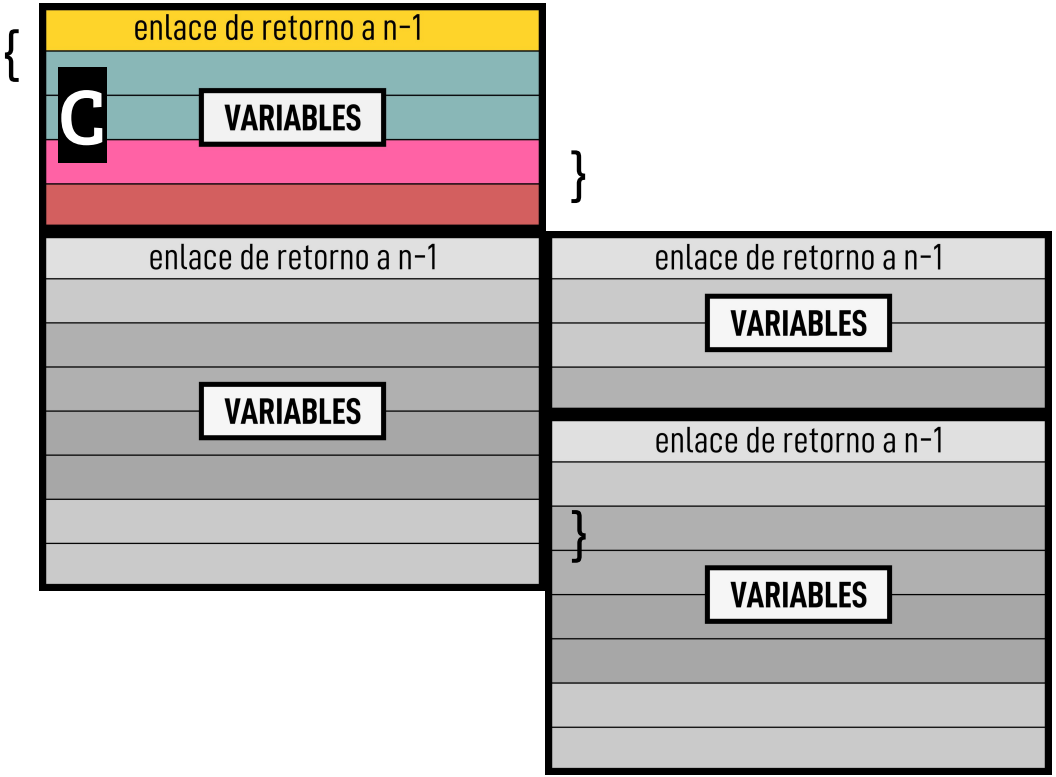
7

A finaliza



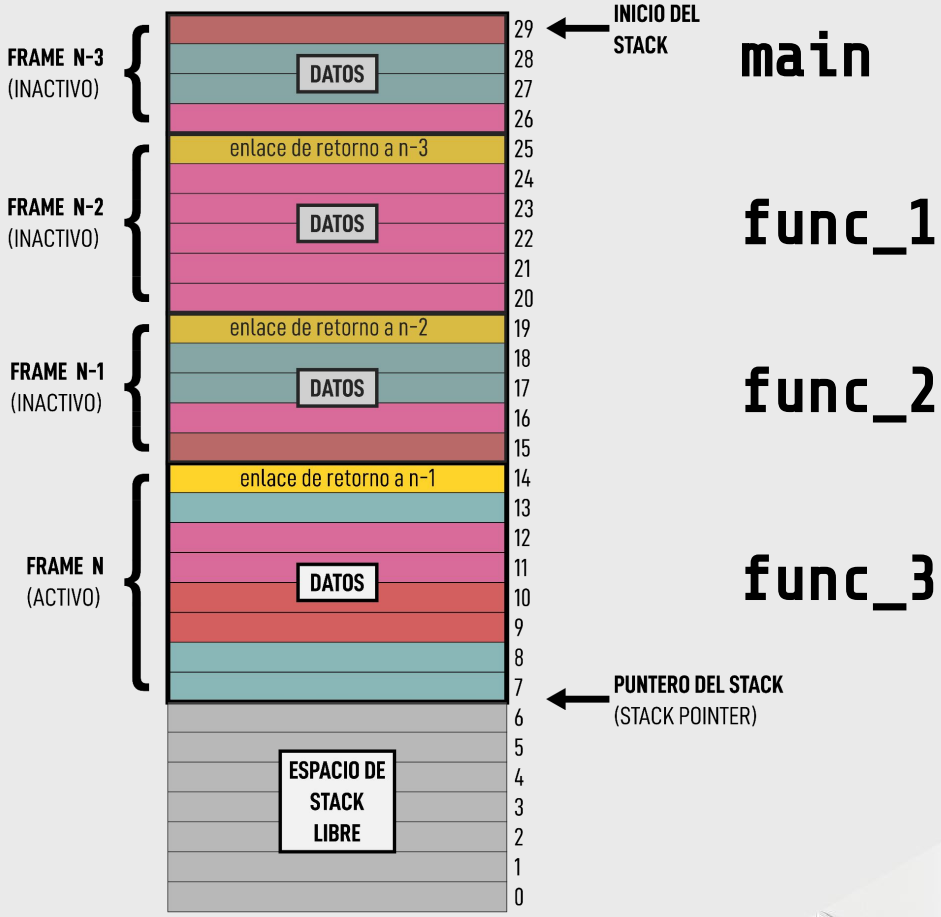
8

llamamos a B


{
enlace de retorno a n-1
C
VARIABLES
}
enlace de retorno a n-1
VARIABLES
enlace de retorno a n-1
VARIABLES
enlace de retorno a n-1
}
VARIABLES

9

B finaliza


FRAME N-3
(INACTIVO)
DATOS
FRAME N-2
(INACTIVO)
enlace de retorno a n-3
DATOS
FRAME N-1
(INACTIVO)
enlace de retorno a n-2
DATOS
FRAME N
(ACTIVO)
enlace de retorno a n-1
DATOS
ESPACIO DE STACK LIBRE
29
28
27
26
25
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
INICIO DEL STACK
PUNTERO DEL STACK
(STACK POINTER)
main
func_1
func_2
func_3

Como el stack reutiliza memoria con cada llamada a función

La memoria *tal vez* venga en 0 la primera vez

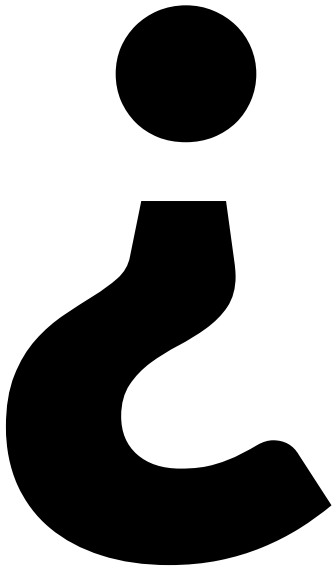
Pero con total seguridad

¡Después no!

¡A la terminal!



{ Lo que tambien nos lleva a... }



Que pasa cuando devolvemos un puntero de una variable automática

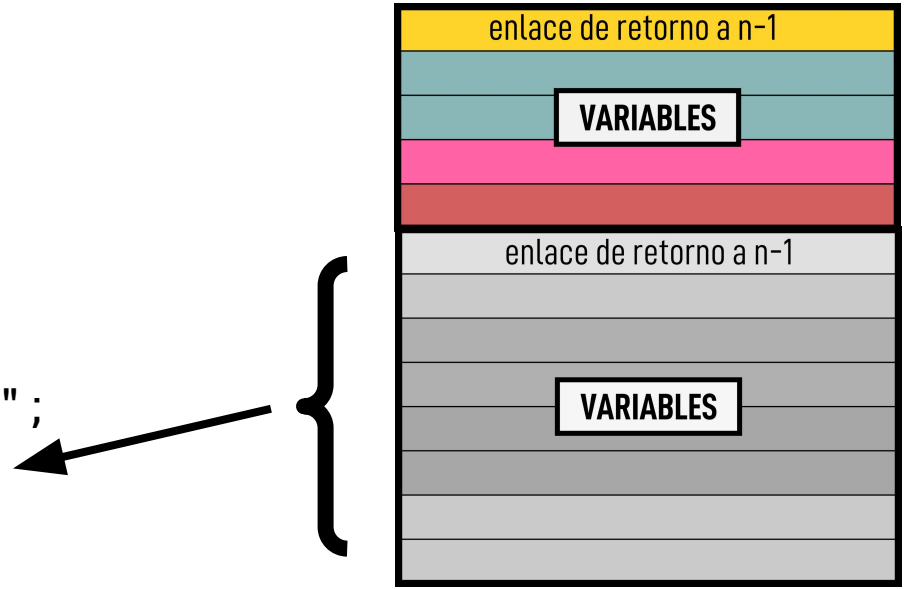


¿Qué veríamos afuera?

```
char* puntero_a_cadena()
{
    char cadena[] = "hola mundo";
    return cadena;
}
```

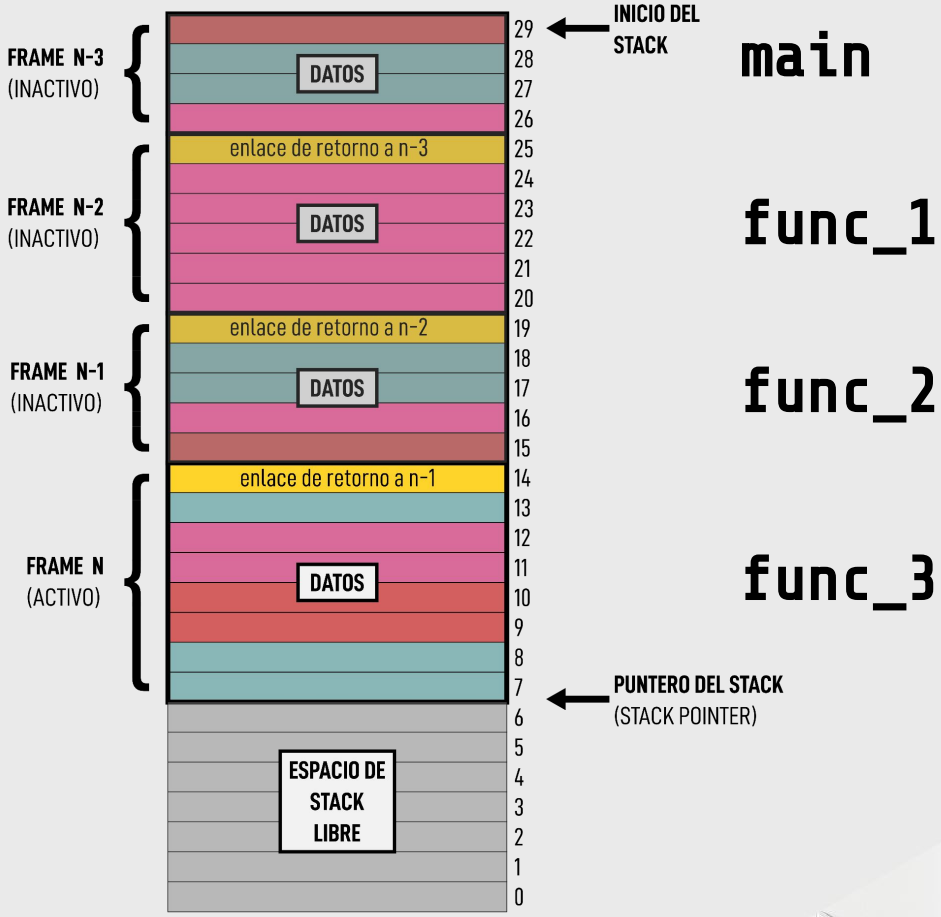
¿Qué veríamos afuera?

```
char* puntero_a_cadena()
{
    char cadena[] = "hola mundo";
    return cadena;
}
```



¡A la terminal!




FRAME N-3
(INACTIVO)
DATOS
FRAME N-2
(INACTIVO)
enlace de retorno a n-3
DATOS
FRAME N-1
(INACTIVO)
enlace de retorno a n-2
DATOS
FRAME N
(ACTIVO)
enlace de retorno a n-1
DATOS
ESPACIO DE STACK LIBRE
29
28
27
26
25
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
INICIO DEL STACK
PUNTERO DEL STACK
(STACK POINTER)
main
func_1
func_2
func_3


¡No nos pertenece!
y cualquier función lo modificará

Y lo pisa cualquier llamada a función, *incluyendo* `printf`


¿Preguntas?

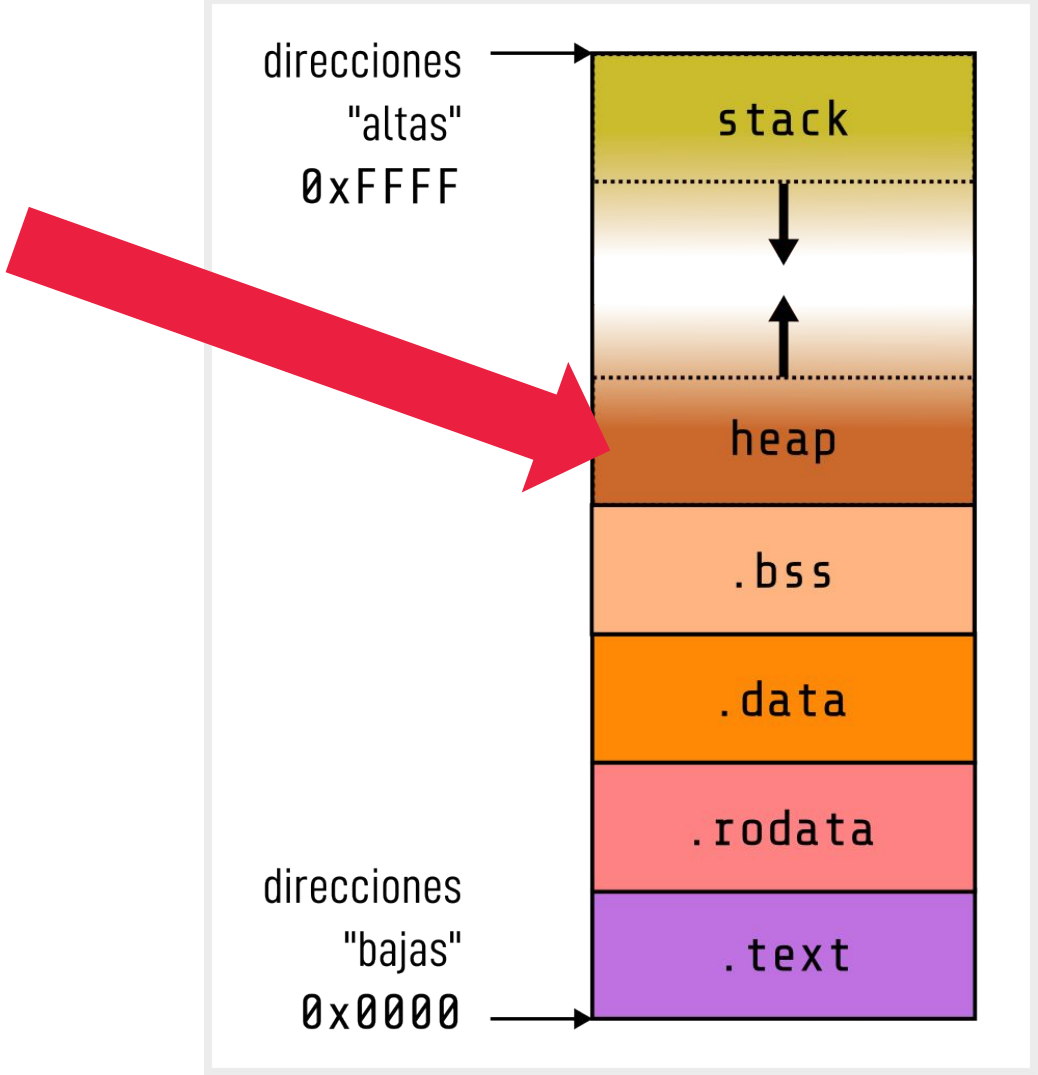
de heap

o manual



UNRN Universidad Nacional de Río Negro

UNRN Universidad Nacional de Río Negro


direcciones
"altas"
0xFFFF
stack
heap
.bss
.data
.rodata
.text
direcciones
"bajas"
0x0000

¿Por qué?

UNRN Universidad Nacional de Río Negro

Flexibilidad

Libera el tamaño de lo que el programa puede procesar a la memoria disponible.

{ Y aunque la memoria dinámica **no es** *estrictamente* necesaria }

Gestión manual

Es necesario, *personalmente*,
pedir y **devolver** la memoria

¿Cómo se usa?

UNRN Universidad Nacional de **Río Negro**

en stdlib.h

```
void* malloc(size_t size);
```

El bloque pedido



```
void* malloc(size_t size);
```

¿lo qué?



```
void* malloc(size_t size);
```



`void` no era

sin retorno



`void*`



¡Un puntero!

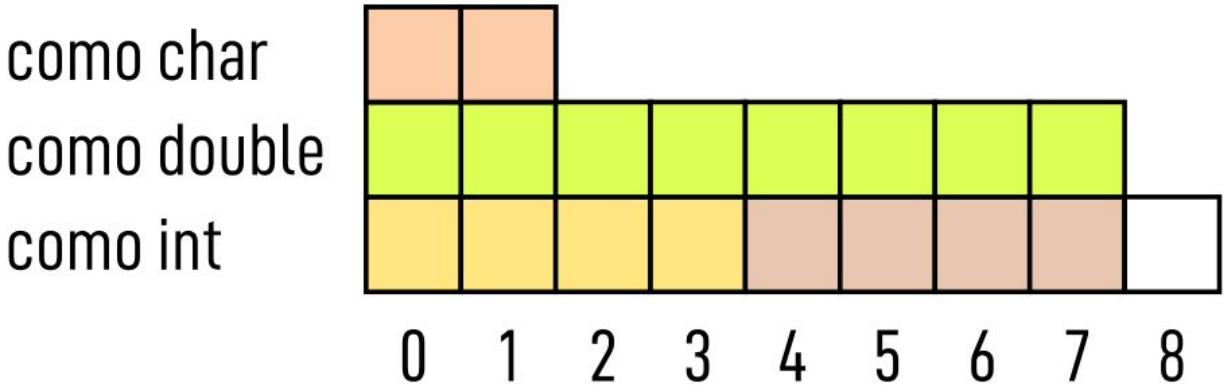
`void*`



`void*`

es **únicamente** la dirección en memoria sin noción de ***'ancho'***

¿Ancho? (en memoria)



*no a escala

`void*` contiene solo la dirección

Por lo que

No es válido un arreglo de void*

```
void arreglo[10];
```

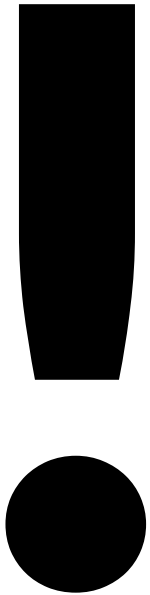
Tampoco es válido

```
void* arreglo = ...;
arreglo[n] = 10
a = arreglo[n]
```

```
          char*
void* ≠ int*
          double*
```



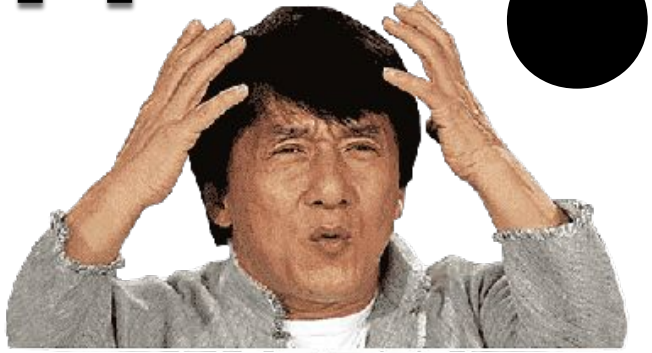
La aritmética de punteros no funciona

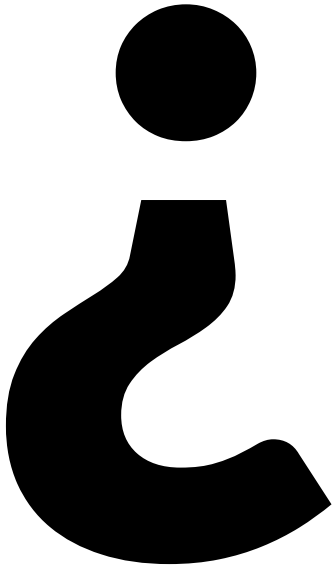



¿Preguntas?

Pero entonces

¿¿Pero cómo se usa??





¿Cómo indicamos el ancho?



Con ejemplo simple

```
int numero = 10;
void* ptr = &numero;
```

¿podemos desreferenciar `ptr`?

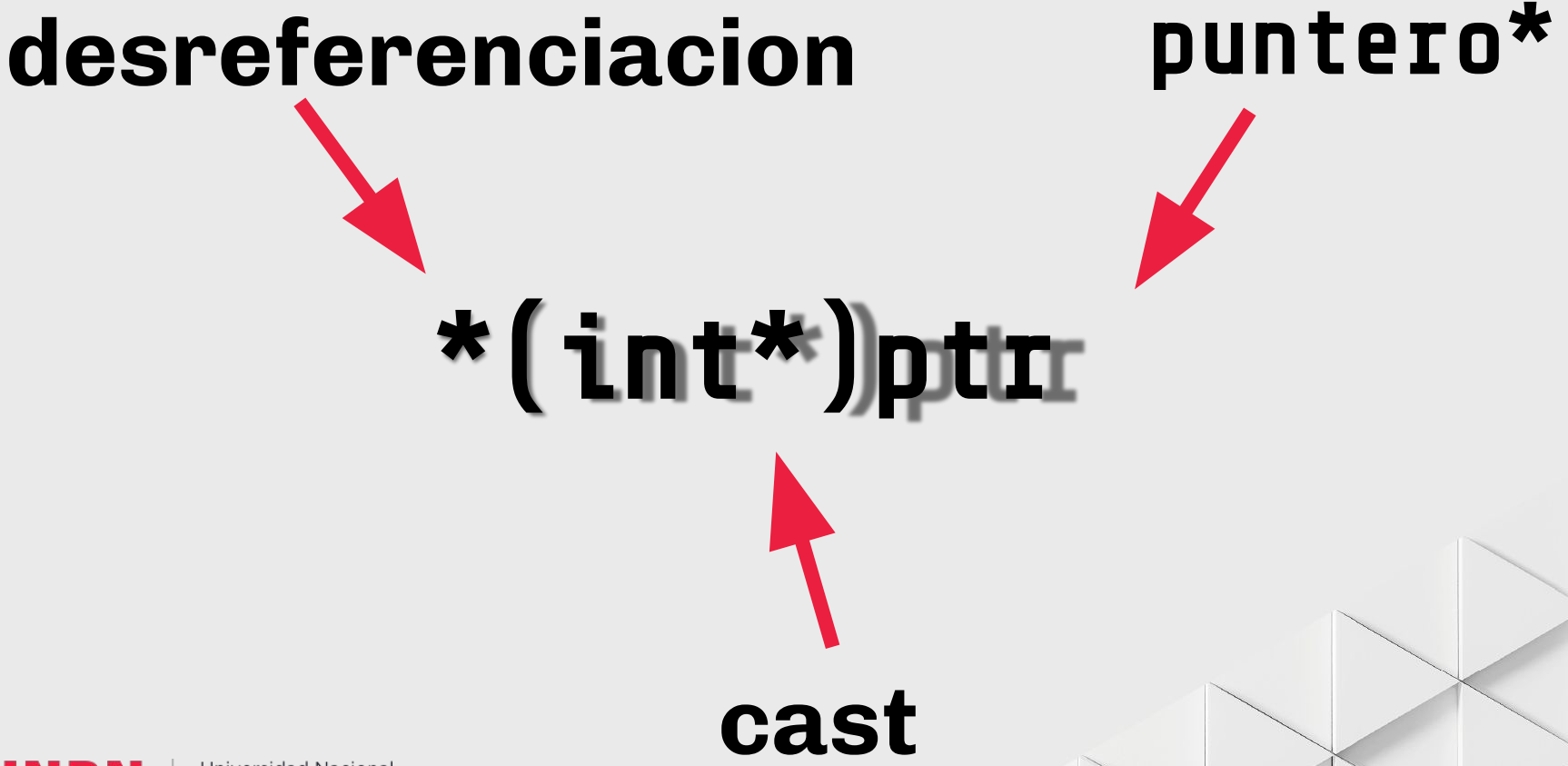
```
int numero = 10;
void* ptr = &numero;
```

```c
int numero = 10;
void* ptr = &numero;

printf("Es %d\n",  *ptr);
```




casts


desreferenciacion
puntero*
(int)ptr
cast

Con una variable intermedia

```
int numero = 10;
void* ptr = &numero;
int* int_ptr = (int*)ptr;

printf("Es %d\n", *int_ptr );
```

¡Ahora sí!

O todo junto

```
int numero = 10;
void* ptr = &numero;

printf("Es %d\n",  *(int*)ptr );
```


¿Preguntas?

void solo es nada

pero

void* es cualquier cosa


¿Preguntas?

Efectos en las funciones

Si una función trabaja con `void*`, no se espera modificar el contenido


¿Preguntas?

Volviendo al `malloc`

UNRN Universidad Nacional de Río Negro

¿Cómo se usa?

UNRN Universidad Nacional de Río Negro

un ejemplo de uso (para un valor individual)

```
char *cadena;
cadena = (char*) malloc(sizeof(char));
```



cast



el tamaño individual

un ejemplo de uso (para un arreglo)

que podemos multiplicar

```
char *cadena;
cadena = (char*) malloc(sizeof(char) * cantidad);
```

cast

el tamaño individual

Después, a usarlo como un arreglo más

El cast explícito es opcional

```
char *cadena = malloc(sizeof(char) * cantidad);
```



cast

Históricamente **ayudaba con errores al no agregar** `#include`

{ Lo que sale del `malloc` no está garantizado en `0` }


¿Preguntas?

¿Puede fallar?

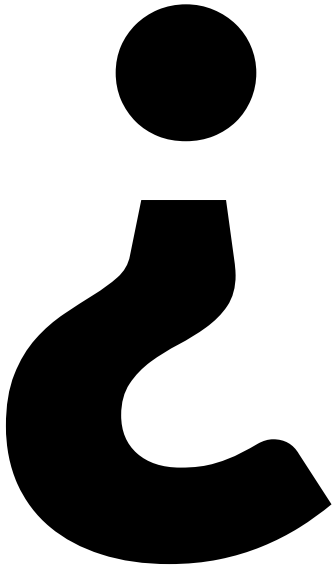

¡Sí!

`malloc` puede no tener memoria para dar

En tal caso, el puntero apunta a NULL

Completando el ejemplo

```
char *cadena = malloc(sizeof(char) * cantidad);
if (cadena == NULL)
{
  abort(); //Oops, nos quedamos sin memoria!
}
//magic happens, le damos uso
```



Si le damos un tamaño 0




¿Preguntas?

Consideraciones de uso

UNRN Universidad Nacional de Río Negro

Sí sabemos el tamaño de las cosas antes de compilar

Mejor un arreglo común

Y *no es muy útil* para valores individuales

Ya que es MUY importante

Liberar la memoria



```
void free(void* ptr);
```

acepta el puntero ‘casteado’

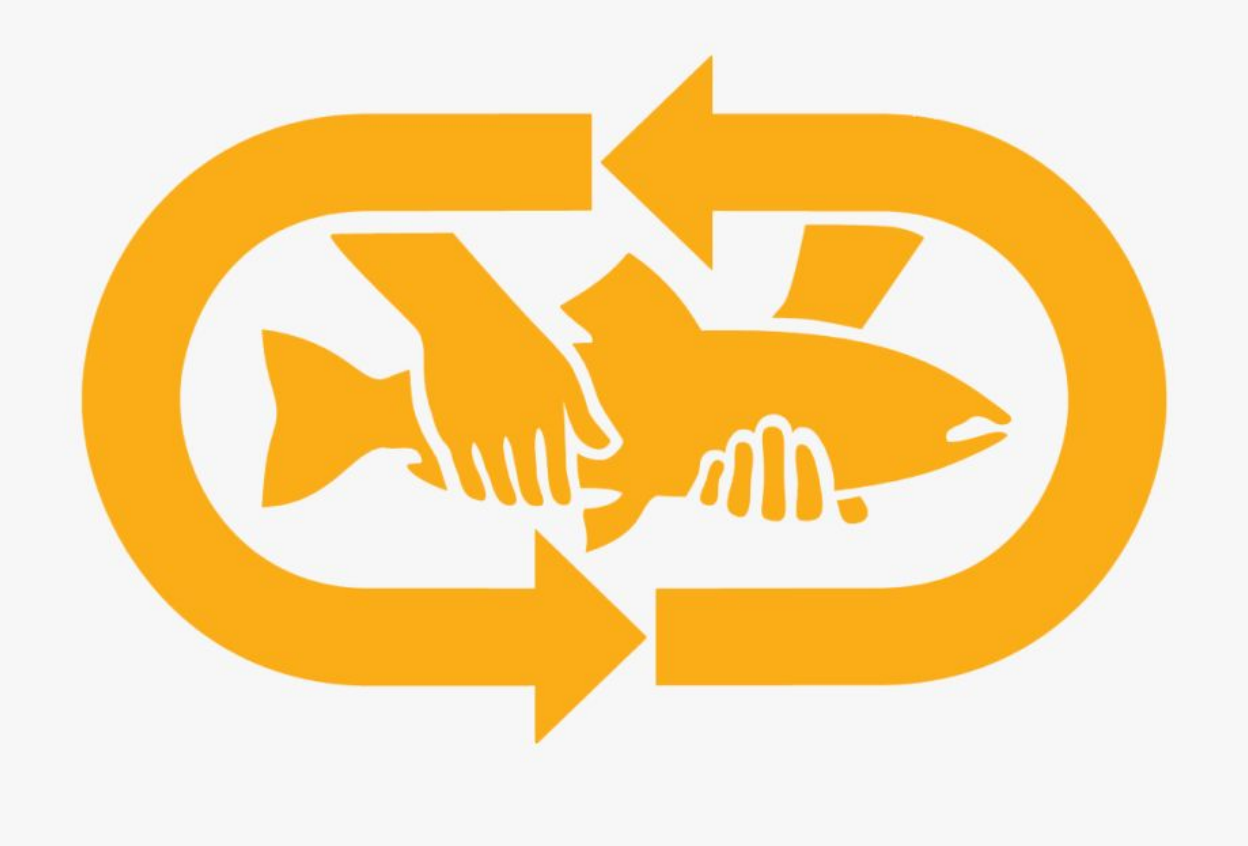


```
void free(void* ptr);
```

Es muy recomendable ‘destruir’ el puntero Asignándole NULL

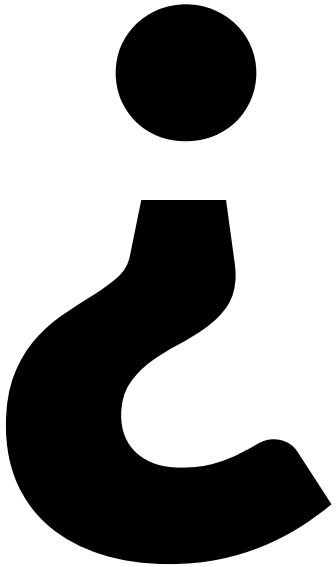
Luego de liberar memoria, asignar `NULL` al puntero

Catch and release!



Completando el ejemplo

```
char* cadena = malloc(sizeof(char) * cantidad);
if (cadena == NULL)
{
  abort();//Oops, nos quedamos sin memoria!
}
//magic happens, le damos uso
free(cadena);
```



Que pasa si...



```
void free(NULL);
```


Kaboom?

Nope, no pasa nada

Nope, *no pasa nada*

pero esto puede ser un problema

Si estábamos esperando liberar memoria, y *la perdimos* por el camino

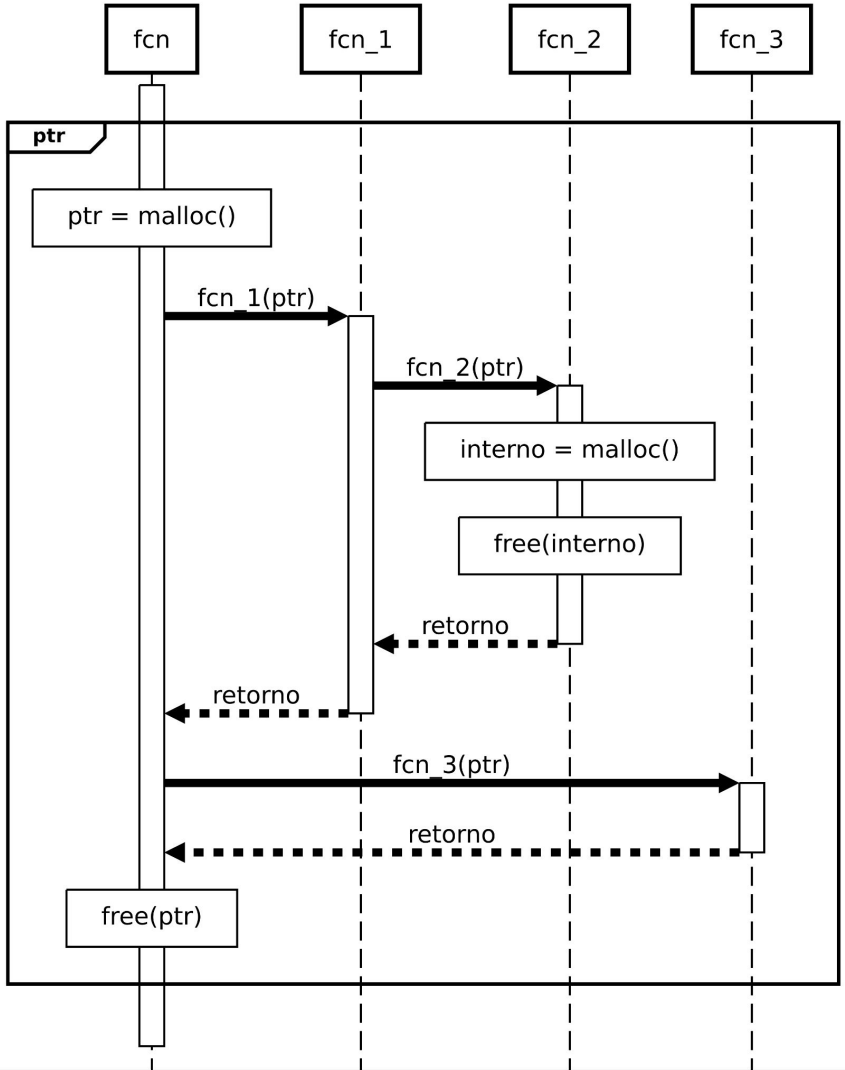

¿Preguntas?

Para hacer esta tarea mas facil

UNRN Universidad Nacional de Río Negro

Por ello,

Pidan y liberen memoria al mismo *nivel**


fcn
fcn_1
fcn_2
fcn_3
ptr
ptr = malloc()
fcn_1(ptr)
fcn_2(ptr)
interno = malloc()
free(interno)
retorno
retorno
fcn_3(ptr)
retorno
free(ptr)

Esto significa que:

18

La memoria se pide en un lugar[1] diferente a donde se usa

19

La memoria se libera en el mismo lugar[1] donde se pide

Ser ordenados ayuda a liberar todo lo pedido

continuará...



Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar